

9 Advanced entity association mappings

This chapter covers

- Applying mapping through one-to-one entity associations
- Using one-to-many mapping options
- Creating many-to-many and ternary entity relationships
- Working with entity associations with maps

In the previous chapter, we demonstrated a unidirectional *many-to-one* association, made it bidirectional, and finally enabled transitive state changes with cascading options. One reason we are discussing more advanced entity mappings in a separate chapter is that we consider quite a few of them rare or at least optional. It's possible to only use component mappings and many-to-one (occasionally *one-to-one*) entity associations. You can write a sophisticated application without ever mapping a collection! We've demonstrated the particular benefits to be gained from collection mappings in the previous chapter, and the rules for when a collection mapping is appropriate also apply to all the examples in this chapter. Always make sure you actually need a collection before attempting a complex collection mapping.

We'll start with mappings that don't involve collections: one-to-one entity associations.

Major new features in JPA 2

Many-to-one and *one-to-one* associations may now be mapped with an intermediate join/link table.

Embeddable component classes may have unidirectional associations to entities, even many-valued with collections.

9.1 One-to-one associations

We argued in section 6.2 that the relationships between `User` and `Address` (the user has a `billingAddress`, `homeAddress`, and `shippingAddress`) are best represented with an `@Embeddable` component mapping. This is usually the simplest way to represent one-to-one rela-

tionships because the lifecycle is typically dependent in such a case. It's either aggregation or composition in UML.

What about using a dedicated `Address` table and mapping both `User` and `Address` as entities? One benefit of this model is that shared references are possible—another entity class (let's say `Shipment`) can also have a reference to a particular `Address` instance. If a `User` also has a reference to this instance as their `shippingAddress`, the `Address` instance has to support shared references and needs its own identity.

In this case, `User` and `Address` classes have a true one-to-one association. Look at the revised class diagram in figure 9.1.

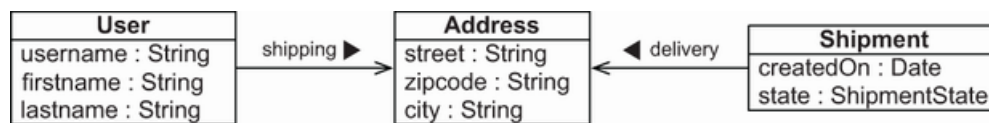


Figure 9.1 `Address` as an entity with two associations, supporting shared references

We are working on the `CaveatEmptor` application, and we need to map the entities from figure 9.1. There are several possible mappings for one-to-one associations. The first strategy we'll consider is a shared primary key value.

NOTE To be able to execute the examples from the source code, you'll first need to run the `Ch09.sql` script.

9.1.1 Sharing a primary key

Rows in two tables related by a primary key association share the same primary key values. If each user has exactly one shipping address, then the approach would be that the `User` has the same primary key value as the (shipping) `Address`. The main difficulty with this approach is ensuring that associated instances are assigned the same primary key value when the instances are saved.

Before we look at this problem, let's create the basic mapping. The `Address` class is now a standalone entity; it's no longer a component. The source code that follows can be found in the `onetoone-sharedprimarykey` folder.

Listing 9.1 `Address` class as a standalone entity

```
Path: onetoone-sharedprimarykey/src/main/java/com/manning/javapersiste
➡ /ch09/onetoone/sharedprimarykey/Address.java
```

```

\1
public class Address {
    @Id
    @GeneratedValue(generator = Constants.ID_GENERATOR)
    private Long id;
    @NotNull
    private String street;
    @NotNull
    private String zipcode;
    @NotNull
    private String city;
    // . . .
}

```

The `User` class is also an entity with the `shippingAddress` association property. We'll introduce two new annotations here: `@OneToOne` and `@PrimaryKeyJoinColumn`.

`@OneToOne` does what you'd expect: it's required to mark an entity-valued property as a one-to-one association. We'll require that a `User` has an `Address` with the `optional=false` clause. We'll force the cascading of changes from `User` to `Address` with the `cascade = CascadeType.ALL` clause. The `@PrimaryKeyJoinColumn` annotation selects the shared primary key strategy we'd like to map.

Listing 9.2 `User` entity and `shippingAddress` association

Path: onetoone-sharedprimarykey/src/main/java/com/manning/javapersistence
 ➡ /ch09/onetoone/sharedprimarykey/User.java

```

@Entity
@Table(name = "USERS")
public class User {
    @Id
    private Long id;

    private String username;

    @OneToOne(
        fetch = FetchType.LAZY,
        optional = false,
        cascade = CascadeType.ALL
    )
    @PrimaryKeyJoinColumn
    private Address shippingAddress;

    public User() {
    }
}

```

①
 ②
 ③
 ④
 ⑤
 ⑥

```

        public User(Long id, String username) {
            this.id = id;
            this.username = username;
        }
        // . . .
    }

```

- Ⓐ For the `User`, we don't declare an identifier generator. As mentioned in section 5.2.4, this is one of the rare cases where we'll use an *application-assigned* identifier value.
- Ⓑ The relationship between `User` and `Address` is one-to-one.
- Ⓒ As usual, we should prefer the lazy-loading strategy, so we override the default `FetchType.EAGER` with `LAZY`.
- Ⓓ The `optional=false` switch specifies that a `User` must have a `shippingAddress`.
- Ⓔ The Hibernate-generated database schema reflects this with a foreign key constraint. Any change here must be cascaded to `Address`. The primary key of the `USERS` table also has a foreign key constraint referencing the primary key of the `ADDRESS` table. See the tables in figure 9.2.
- Ⓕ Using `@PrimaryKeyJoinColumn` makes this a unidirectional shared primary key one-to-one association mapping, from `User` to `Address`.
- Ⓖ The constructor design weakly enforces this: the public API of the class requires an identifier value to create an instance.

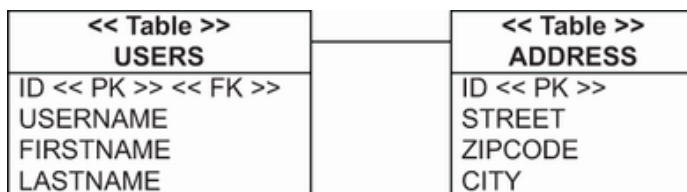


Figure 9.2 The `USERS` table has a foreign key constraint on its primary key.

For some of the examples in this chapter, we'll need to make a few changes to our usual configuration for the tests, as the execution will need to be transactional. The `SpringDataConfiguration` class will require more annotations:

```

Path: onetoone-sharedprimarykey/src/test/java/com/manning/javapersiste
➡ /ch09/configuration/onetoone/sharedprimarykey
➡ /SpringDataConfiguration.java

```

```

@Configuration
@EnableTransactionManagement
@ComponentScan(basePackages = "com.manning.javapersistence.ch09.*")
@EnableJpaRepositories("com.manning.javapersistence.ch09.repositories.
                        onetoone.sharedprimarykey")
public class SpringDataConfiguration {
    // . . .
}

```

- Ⓐ `@Configuration` specifies that this class declares one or more bean definitions to be used by the Spring container.
- Ⓑ `@EnableTransactionManagement` enables the transaction management capabilities of Spring through annotations.
- Ⓒ We'll need to execute a few operations in a transactional way to test the code from this chapter. `@ComponentScan` requires Spring to scan the package provided as an argument, and its subpackages, for components.
- Ⓓ `@EnableJpaRepositories` scans the indicated package to find Spring Data repositories.

We'll isolate the operations against the database in a dedicated `TestService` class:

```

Path: onetoone-sharedprimarykey/src/test/java/com/manning/javapersistence
➡ /ch09/onetoone/sharedprimarykey/TestService.java

```

```

@Service Ⓐ
public class TestService {
    @Autowired Ⓑ
    private UserRepository userRepository; Ⓑ

    @Autowired Ⓑ
    private AddressRepository addressRepository; Ⓑ

    @Transactional Ⓒ
    public void storeLoadEntities() { Ⓒ
        // . . .
    }
}

```

- Ⓐ The `TestService` class is annotated as `@Service` to allow Spring to automatically create a bean, later to be injected in the effective test. Remember that in the `SpringDataConfiguration` class, we scan the `com.manning.javapersistence.ch09` package and its subpackages for components.
- Ⓑ Inject two repository beans.

© Define the `storeLoadEntities` method, annotating it with `@Transactional`. The operations we'll need to execute against the database need to be transactional, and we'll let Spring control this.

The testing class will differ from the previously presented ones, as it will delegate to the `TestService` class. This will allow us to keep the transactional operations isolated in their own methods and to call those methods from the test.

```
Path: onetoone-sharedprimarykey/src/test/java/com/manning/javapersistence
➡ /ch09/onetoone/sharedprimarykey/AdvancedMappingSpringDataJPATest.java
```

```
@ExtendWith(SpringExtension.class)
@ContextConfiguration(classes = {SpringDataConfiguration.class})
public class AdvancedMappingSpringDataJPATest {

    @Autowired
    private TestService testService;

    @Test
    void testStoreLoadEntities() {

        testService.storeLoadEntities();

    }
}
```

The JPA specification doesn't include a standardized method for dealing with the problem of shared primary key generation. This means we're responsible for setting the identifier value of a `User` instance correctly before we save it to the identifier value of the linked `Address` instance:

```
Path: onetoone-sharedprimarykey/src/test/java/com/manning/javapersistence
➡ /ch09/onetoone/sharedprimarykey/TestService.java
```

```
Address address =
    new Address("Flowers Street", "01246", "Boston");
addressRepository.save(address); ①

User john = new User(address.getId(), "John Smith"); ②
john.setShippingAddress(address);

userRepository.save(john); ③
```

① Persist the `Address`.

② Take its generated identifier value and set it on `User`.

There are three problems with the mapping and code:

- We have to remember that the `Address` must be saved first and then get its identifier value. This is only possible if the `Address` entity has an identifier generator that produces values on `save()` before the `INSERT`, as we discussed in section 5.2.5. Otherwise, `someAddress.getId()` returns `null`, and we can't manually set the identifier value of the `User`.
- Lazy loading with proxies only works if the association is non-optional. This is often a surprise for developers new to JPA. The default for `@OneToOne` is `FetchType.EAGER`: when Hibernate or Spring Data JPA using Hibernate loads a `User`, it loads the `shippingAddress` right away. Conceptually, lazy loading with proxies only makes sense if Hibernate knows that there is a linked `shippingAddress`. If the property were nullable, Hibernate would have to check in the database whether the property value is `NULL` by querying the `ADDRESS` table. If we have to check the database, we might as well load the value right away, because there would be no benefit in using a proxy.
- The one-to-one association is unidirectional; sometimes we need bidirectional navigation.

The first problem has no other solution. In the preceding example we are doing exactly this: saving the `Address`, getting its primary key, and manually setting it as the identifier value of the `User`. This is one of the reasons we should always prefer identifier generators capable of producing values before any SQL `INSERT`.

A `@OneToOne(optional=true)` association doesn't support lazy loading with proxies. This is consistent with the JPA specification.

`FetchType.LAZY` is a hint for the persistence provider, not a requirement. We could get lazy loading of nullable `@OneToOne` with bytecode instrumentation, as you'll see in section 12.1.3.

As for the last problem, if we make the association bidirectional (where the `Address` references the `User` and the `User` references the `Address`), we can also use a special Hibernate-only identifier generator to help with assigning key values.

9.1.2 The foreign primary key generator

A bidirectional mapping always requires a `mappedBy` side. We'll pick the `User` side—this is a matter of taste and perhaps other, secondary requirements. (The source code that follows can be found in the `onetoone-foreigngenerator` folder.)

```
Path: onetoone-foreigngenerator/src/main/java/com/manning/javapersiste
➡ /ch09/onetoone/foreigngenerator/User.java
```

```
@Entity
@Table(name = "USERS")
public class User {
    @Id
    @GeneratedValue(generator = Constants.ID_GENERATOR)
    private Long id;

    private String username;

    @OneToOne(
        mappedBy = "user",
        cascade = CascadeType.PERSIST
    )
    private Address shippingAddress;
    // . . .
}
```

We have added the `mappedBy` option, telling Hibernate or Spring Data JPA using Hibernate that the lower-level details are now mapped by “the property on the other side,” named `user`. As a convenience, we enable `CascadeType.PERSIST`; transitive persistence will make it easier to save the instances in the right order. When we make the `User` persistent, Hibernate makes the `shippingAddress` persistent and generates the identifier for the primary key automatically.

Next, let’s look at the “other side”: the `Address`. We’ll use the `@GenericGenerator` on the identifier property to define a special-purpose primary key value generator with the Hibernate-only `foreign` strategy. We didn’t mention this generator in the overview in section 5.2.5 because the shared primary key one-to-one association is its only use case. When persisting an instance of `Address`, this special generator grabs the value of the `user` property and takes the identifier value of the referenced entity instance, the `User`.

Listing 9.3 `Address` has the special foreign key generator

```
Path: onetoone-foreigngenerator/src/main/java/com/manning/javapersiste
➡ /ch09/onetoone/foreigngenerator/Address.java
```

```
@Entity
public class Address {
    @Id
    @GeneratedValue(generator = "addressKeyGenerator")
    @org.hibernate.annotations.GenericGenerator(
        name = "addressKeyGenerator",
```



```

        strategy = "foreign",
        parameters =
            @org.hibernate.annotations.Parameter(
                name = "property", value = "user"
            )
    )
    private Long id;

    // . . .

    @OneToOne(optional = false)           ③
    @PrimaryKeyJoinColumn                 ④
    private User user;

    public Address() {
    }

    public Address(User user) {           ①
        this.user = user;
    }

    public Address(User user, String street,           ②
                    String zipcode, String city) {
        this.user = user;
        this.street = street;
        this.zipcode = zipcode;
        this.city = city;
    }
    // . . .
}

```

- ① With the `@GenericGenerator` annotation, when we persist an instance of `Address`, this special generator grabs the value of the `user` property and takes the identifier value of the referenced entity instance, the `User`.
- ② The `@OneToOne` mapping is set to `optional=false`, so an `Address` must have a reference to a `User`.
- ③ The `user` property is marked as a shared primary key entity association with the `@PrimaryKeyJoinColumn` annotation.
- ④ The public constructors of `Address` now require a `User` instance. The foreign key constraint reflecting `optional=false` is now on the primary key column of the `ADDRESS` table, as you can see in the schema in figure 9.3.

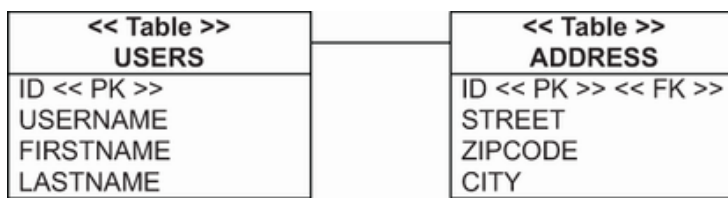


Figure 9.3 The ADDRESS table has a foreign key constraint on its primary key.

Thanks to this new code, we no longer have to call `address.getId()` or `user.getId()` in our unit of work. Storing data is simplified:

```
Path: onetoone-foreigngenerator/src/test/java/com/manning/javapersistence
➡ /ch09/onetoone/foreigngenerator/AdvancedMappingJPATest.java
```

```
User john = new User("John Smith");
Address address =
    new Address(
        john,
        "Flowers Street", "01246", "Boston"
    );
john.setShippingAddress(address);
userRepository.save(john);
```

Ⓐ We must link both sides of a bidirectional entity association. Note that with this mapping, we won’t get lazy loading of `User#shippingAddress` (it’s optional/nullable), but we can load `Address#user` on demand with proxies (it’s non-optional).

Ⓑ When we persist the user, we’ll get the transitive persistence of `shippingAddress`.

Shared primary key one-to-one associations are relatively rare. Instead, we’ll often map a “to-one” association with a foreign key column and a unique constraint.

9.1.3 Using a foreign key join column

Instead of sharing a primary key, two rows can have a relationship based on a simple additional foreign key column. One table has a foreign key column that references the primary key of the associated table. (The source and target of this foreign key constraint can even be the same table: we call this a *self-referencing relationship*.) The source code that follows can be found in the `onetoone-foreignkey` folder.

Let’s change the mapping for `User#shippingAddress`. Instead of having a shared primary key, we’ll now add a `SHIPPINGADDRESS_ID` column in the `USERS` table. This column has a `UNIQUE` constraint, so no

two users can reference the same shipping address. Look at the schema in figure 9.4.



Figure 9.4 A one-to-one join column association between the `USERS` and `ADDRESS` tables

The `Address` is a regular entity class, like the first one we demonstrated in this chapter, in listing 9.1. The `User` entity class has the `shippingAddress` property, implementing this unidirectional association.

We should enable lazy loading for this `User – Address` association. Unlike with shared primary keys, we don't have a problem with lazy loading here: when a row of the `USERS` table has been loaded, it contains the value of the `SHIPPINGADDRESS_ID` column. Hibernate or Spring Data using Hibernate, therefore, knows whether an `ADDRESS` row is present, and a proxy can be used to load the `Address` instance on demand.

```
Path: onetoone-foreignkey/src/main/java/com/manning/javapersistence
➡ /ch09/onetoone/foreignkey/User.java
```

```
@Entity
@Table(name = "USERS")
public class User {
    @Id
    @GeneratedValue(generator = Constants.ID_GENERATOR)
    private Long id;

    @OneToOne(
        fetch = FetchType.LAZY,
        optional = false,
        cascade = CascadeType.PERSIST
    )
    @JoinColumn(unique = true)
    private Address shippingAddress;
    // . . .
}
```

Ⓐ We don't need any special identifier generators or primary key assignments; we'll make sure the `shippingAddress` is not null.

Ⓑ Instead of `@PrimaryKeyJoinColumn`, we apply the regular `@JoinColumn`, which will default to `SHIPPINGADDRESS_ID`. If you're more familiar with SQL than JPA, it helps to think "foreign key column" every time you see `@JoinColumn` in a mapping.

In the mapping, we set `optional=false`, so the user must have a shipping address. This won't affect the loading behavior but it is a logical consequence of the `unique=true` setting on the `@JoinColumn`. This setting adds a unique constraint to the generated SQL schema. If the values of the `SHIPPINGADDRESS_ID` column must be unique for all users, only one user could possibly have "no shipping address." Hence, nullable unique columns typically aren't meaningful.

Creating, linking, and storing instances is straightforward:

```
Path: onetoone-foreignkey/src/test/java/com/manning/javapersistence
➡ /ch09/onetoone/foreignkey/AdvancedMappingSpringDataJPATest.java
```

```
User john = new User("John Smith");
Address address = new Address("Flowers Street", "01246", "Boston");
john.setShippingAddress(address);
userRepository.save(john);
```

Ⓐ Create the link between the user and the address.

Ⓑ When we save `john`, we'll transitively save the `address`.

We've now completed two basic one-to-one association mappings: the first with a shared primary key, and the second with a foreign key reference and a unique column constraint. The last option we want to discuss is a bit more exotic: mapping a one-to-one association with the help of an additional table.

9.1.4 Using a join table

You've probably noticed that nullable columns can be problematic. Sometimes a better solution for optional values is to use an intermediate table, which contains a row if a link is present and doesn't if it's not.

Let's consider the `Shipment` entity in `CaveatEmptor` and discuss its purpose. Sellers and buyers interact in `CaveatEmptor` by starting and bidding on auctions. Shipping goods seems outside the scope of the application; the seller and the buyer agree on a method of shipment and payment after the auction ends. They can do this offline, outside of `CaveatEmptor`.

On the other hand, we could offer an escrow service in CaveatEmptor. Sellers would use this service to create a trackable shipment once the auction ends. The buyer would pay the price of the auction item to a trustee (us), and we'd inform the seller that the money is available. Once the shipment arrives and the buyer accepts it, we'd transfer the money to the seller.

If you've ever participated in an online auction of significant value, you've probably used such an escrow service. But we want more in CaveatEmptor: not only will we provide trust services for completed auctions, but we also allow users to create trackable and trusted shipments for any deals they make outside an auction, outside CaveatEmptor. This scenario calls for a `Shipment` entity with an optional one-to-one association to `Item`. The class diagram for this domain model is shown in figure 9.5.

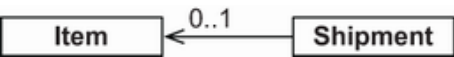


Figure 9.5 `Shipment` has an optional link with an auction `Item`.

NOTE We briefly considered abandoning the CaveatEmptor example for this section because we couldn't find a natural scenario that requires optional one-to-one associations. If this escrow example seems contrived, consider the equivalent problem of assigning employees to workstations. This is also an optional one-to-one relationship.

In the database schema, we'll add an intermediate link table called `ITEM_SHIPMENT`. A row in this table represents a `Shipment` made in the context of an auction. Figure 9.6 shows the tables.



Figure 9.6 The intermediate table links items and shipments.

Note how the schema enforces uniqueness and the one-to-one relationship: the primary key of `ITEM_SHIPMENT` is the `SHIPMENT_ID` column, and the `ITEM_ID` column is unique. An item can therefore be in only one shipment. Of course, that also means a shipment can contain only one item.

We'll map this model with a `@OneToOne` annotation in the `Shipment` entity class. The source code that follows can be found in the `onetoone-jointable` folder.

Path: onetoone-jointable/src/main/java/com/manning/javapersistence
➡ /ch09/onetoone/jointable/Shipment.java

```
@Entity
public class Shipment {
    // . . .
    @OneToOne(fetch = FetchType.LAZY)           ①
    @JoinTable(
        name = "ITEM_SHIPMENT",                 ②
        joinColumns =
            @JoinColumn(name = "SHIPMENT_ID"),   ③
        inverseJoinColumns =
            @JoinColumn(name = "ITEM_ID",        ④
                        nullable = false,
                        unique = true)           ⑤
    )
    private Item auction;

    // . . .
}
```

① Lazy loading has been enabled, with a twist: when Hibernate or Spring Data JPA using Hibernate loads a `Shipment`, it queries both the `SHIPMENT` and the `ITEM_SHIPMENT` join table. Hibernate has to know if a link to an `Item` is present before it can use a proxy. It does that in one outer join SQL query, so we won't see any extra SQL statements. If there is a row in `ITEM_SHIPMENT`, Hibernate uses an `Item` placeholder.

② The `@JoinTable` annotation is new; we always have to specify the name of the intermediate table. This mapping effectively hides the join table; there is no corresponding Java class. The annotation defines the column names of the `ITEM_SHIPMENT` table.

③ The join column is `SHIPMENT_ID` (it would default to `ID`).

④ The inverse join column is `ITEM_ID` (it would default to `AUCTION_ID`).

⑤ Hibernate generates the `UNIQUE` constraint on the `ITEM_ID` column in the schema. Hibernate also generates the appropriate foreign key constraints on the columns of the join table.

Here we store a `Shipment` without `Items` and another linked to a single `Item`:

Path: onetoone-jointable/src/test/java/com/manning/javapersistence
➡ /ch09/onetoone/jointable/AdvancedMappingSpringDataJPATest.java

```
Shipment shipment = new Shipment();
shipmentRepository.save(shipment);
Item item = new Item("Foo");
itemRepository.save(item);
Shipment auctionShipment = new Shipment(item);
shipmentRepository.save(auctionShipment);
```

This completes our discussion of one-to-one association mappings. To summarize, you should use a shared primary key association if one of the two entities is always stored before the other and can act as the primary key source. Use a foreign key association in all other cases, or a hidden intermediate join table when the one-to-one association is optional.

We'll now focus on plural, or *many-valued*, entity associations, beginning with some advanced options for one-to-many.

9.2 One-to-many associations

A *plural entity association* is, by definition, a collection of entity references. We mapped one of these, a one-to-many association, in section 8.3.2. One-to-many associations are the most important kind of entity association involving a collection. We'll go so far as to discourage the use of more complex association styles when a simple bidirectional many-to-one or one-to-many will do the job.

Also, remember that you don't have to map any collection of entities if you don't want to; you can always write an explicit query instead of direct access through iteration. If you decide to map collections of entity references, you have a few options, and we'll analyze some more complex situations now.

9.2.1 Considering one-to-many bags

So far, we have only seen a `@OneToMany` on a `Set`, but it's possible to use a bag mapping for a bidirectional one-to-many association. Why would we do this?

Bags have the most efficient performance characteristics of all the collections we can use for a bidirectional one-to-many entity association. By default, collections in Hibernate are loaded when they're accessed for the first time in the application. Because a bag doesn't have to maintain the index of its elements (like a list) or check for duplicate elements (like a set), we can add new elements to the bag without triggering the loading. This is an important feature if we're going to map a possibly large collection of entity references.

On the other hand, we can't eager-fetch two collections of bag type simultaneously, as the generated `SELECT` queries are unrelated and need to be kept separately. This might happen if `bids` and `images` of an `Item` were one-to-many bags, for example. This is no big loss, because fetching two collections simultaneously always results in a Cartesian product; we want to avoid this kind of operation, whether the collections are bags, sets, or lists. We'll come back to fetching strategies in chapter 12. In general, we'd say that a bag is the best inverse collection for a one-to-many association if it is mapped as a `@OneToMany(mappedBy = "...")`.

To map our bidirectional one-to-many association as a bag, we have to replace the type of the `bids` collection in the `Item` entity with a `Collection` and an `ArrayList` implementation. The mapping for the association between `Item` and `Bid` remains essentially unchanged. (The source code that follows can be found in the `onetomany-bag` folder.)

```
Path: onetomany-bag/src/main/java/com/manning/javapersistence/ch09
➡ /onetomany-bag/Item.java
```

```
@Entity
public class Item {
    / . . .
    @OneToMany(mappedBy = "item")
    private Collection<Bid> bids = new ArrayList<>();
    // . . .
}
```

The `Bid` side with its `@ManyToOne` (which is the “mapped by” side) and even the tables are the same as in section 8.3.1.

A bag also allows duplicate elements, which a set does not:

```
Path: onetomany-bag/src/test/java/com/manning/javapersistence/ch09
➡ /onetomany-bag/AdvancedMappingSpringDataJPATest.java
```

```
Item item = new Item("Foo");
itemRepository.save(item);
Bid someBid = new Bid(new BigDecimal("123.00"), item);
item.addBid(someBid);
item.addBid(someBid);
bidRepository.save(someBid);
assertEquals(2, someItem.getBids().size());
```

It turns out that isn't relevant in this case because *duplicate* means we've added a particular reference to the same `Bid` instance several times. We wouldn't do this in our application code. Even if we added the same refer-

ence several times to this collection, though, Hibernate or Spring Data JPA using Hibernate would ignore it—there is no persistent effect. The side relevant for updates of the database is the `@ManyToOne`, and the relationship is already “mapped by” that side. When we load the `Item`, the collection doesn’t contain the duplicate:

```
Path: onetomany-bag/src/test/java/com/manning/javapersistence/ch09/
➡ onetomany/bag/AdvancedMappingSpringDataJPATest.java
```

```
Item item2 = itemRepository.findItemWithBids(item.getId());
assertEquals(1, item2.getBids().size());
```

As mentioned previously, the advantage of bags is that the collection doesn’t have to be initialized when we add a new element:

```
Path: onetomany-bag/src/test/java/com/manning/javapersistence/ch09/
➡ onetomany/bag/AdvancedMappingSpringDataJPATest.java
```

```
Bid bid = new Bid(new BigDecimal("456.00"), item);
item.addBid(bid);
bidRepository.save(bid);
```

Ⓐ This code example triggers one SQL `SELECT` to load the `Item`. Hibernate still initializes and returns an `Item` proxy with a `SELECT` as soon as we call `item.addBid()`. But as long as we don’t iterate the `Collection`, no more queries are necessary, and an `INSERT` for the new `Bid` will be made without loading all the bids. If the collection is a `Set` or a `List`, Hibernate loads all the elements when we add another element.

We’ll now change the collection to a persistent `List`.

9.2.2 Unidirectional and bidirectional list mappings

If we need a real list to hold the position of the elements in a collection, we have to store that position in an additional column. For a one-to-many mapping, this also means we should change the `Item#bids` property to `List` and initialize the variable with an `ArrayList`. This will be a unidirectional mapping: there will be no other “mapped by” side. The `Bid` will not have a `@ManyToOne` property. For persistent list indexes, we’ll use the annotation `@OrderColumn`. (The source code that follows can be found in the `onetomany-list` folder.)

```
Path: onetomany-list/src/main/java/com/manning/javapersistence/ch09/
➡ onetomany/list/Item.java
```

```

@Entity
public class Item {
    @OneToMany
    @JoinColumn(
        name = "ITEM_ID",
        nullable = false
    )
    @OrderColumn(
        name = "BID_POSITION",    ❶
        nullable = false          ❷
    )
    private List<Bid> bids = new ArrayList<>();
    // . . .
}

```

❶ As mentioned, this is a unidirectional mapping: there is no other “mapped by” side. The `Bid` doesn’t have a `@ManyToOne` property. The `@OrderColumn` annotation will set the name of the index column to be `BID_POSITION`. Otherwise, it would default to `BIDS_ORDER`.

❷ As usual, we should make the column `NOT NULL`.

The database view of the `BID` table, with the join and order columns, is shown in figure 9.7.

BID			
ID	ITEM_ID	BID_POSITION	AMOUNT
1	1	0	99.00
2	1	1	100.00
3	1	2	101.00
4	2	0	4.99

Figure 9.7 The `BID` table contains the `ITEM_ID` (join column) and `BID_POSITION` (order column).

The stored index of each collection starts at zero and is contiguous (there are no gaps). Hibernate or Spring Data JPA using Hibernate will execute potentially many SQL statements when we add, remove, and shift elements of the `List`. We talked about this performance issue in section 8.1.6.

Let’s make this mapping bidirectional, with a `@ManyToOne` property on the `Bid` entity:

Path: onetomany-list/src/main/java/com/manning/javapersistence/ch09/

➡ onetomany/list/Bid.java

```
@Entity
public class Bid {
    // . . .

    @ManyToOne
    @JoinColumn(
        name = "ITEM_ID",
        updatable = false, insertable = false    ❸
    )
    @NotNull
    private Item item;
    // . . .
}
```

❸ The `Item#bids` collection is no longer read-only because Hibernate now has to store the index of each element. If the `Bid#item` side was the owner of the relationship, Hibernate would ignore the collection when storing data and not write the element indexes. We have to map the `@JoinColumn` twice and then disable writing on the `@ManyToOne` side with `updatable=false` and `insertable=false`. Hibernate now considers the collection side when storing data, including the index of each element. The `@ManyToOne` is effectively read-only, as it would be if it had a `mappedBy` attribute.

You probably expected different code—maybe

`@ManyToOne(mappedBy="bids")` and no additional `@JoinColumn` annotation. But `@ManyToOne` doesn't have a `mappedBy` attribute: it's always the "owning" side of the relationship. We'd have to make the other side, `@OneToMany`, the `mappedBy` side.

Finally, the Hibernate schema generator always relies on the `@JoinColumn` of the `@ManyToOne` side. Hence, if we want the correct schema produced, we should add the `@NotNull` on this side or declare `@JoinColumn(nullable=false)`. The generator ignores the `@OneToMany` side and its join column if there is a `@ManyToOne`.

In a real application, we wouldn't map this association with a `List`. Preserving the order of elements in the database seems like a common use case, but it isn't actually very useful: sometimes we'll want to show a list with the highest or newest bid first, or only show bids made by a certain user, or show bids made within a certain time range. None of these operations requires a persistent list index. As mentioned in section 3.2.4, it's best to avoid storing a display order in the database, as the display or-

der may change frequently; keep it flexible with queries instead of using hardcoded mappings. Furthermore, maintaining the index when the application removes, adds, or shifts elements in a list can be expensive and may trigger many SQL statements. Map the foreign key join column with `@ManyToOne`, and drop the collection.

Next we'll work on one more scenario with a one-to-many relationship: an association mapped to an intermediate join table.

9.2.3 Optional one-to-many with a join table

A useful addition to the `Item` class is a `buyer` property. We can then call `someItem.getBuyer()` to access the `User` who made the winning bid. If it's made bidirectional, this association will also help us render a screen that shows all the auctions a particular user has won: we could call `someUser.getBoughtItems()` instead of writing a query.

From the point of view of the `User` class, the association is one-to-many. Figure 9.8 shows the classes and their relationship.



Figure 9.8 The `User`–`Item` “bought” relationship

Why is this association different from the one between `Item` and `Bid`? The multiplicity `0..*` in UML indicates that the reference is optional. This doesn't influence the Java domain model much, but it has consequences for the underlying tables. We expect a `BUYER_ID` foreign key column in the `ITEM` table, but now the column has to be nullable because a user may not have bought a particular `Item` (as long as the auction is still running).

We could accept that the foreign key column can be `NULL` and apply additional constraints: “Allowed to be `NULL` only if the auction end time hasn't been reached or if no bid has been made.” However, we always try to avoid nullable columns in a relational database schema. Unknown information degrades the quality of the data we store. Tuples represent propositions that are true, and we can't assert something we don't know. Moreover, in practice, many developers and DBAs don't create the right constraints and rely on often buggy application code to provide data integrity.

An optional entity association, be it one-to-one or one-to-many, is best represented in an SQL database with a join table. Figure 9.9 shows an example schema.

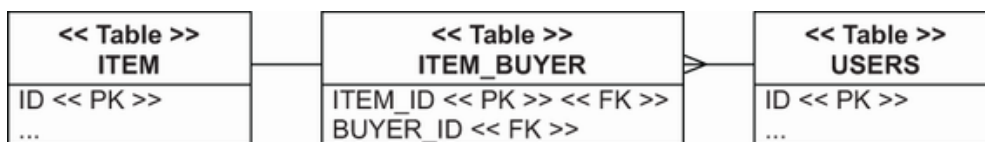


Figure 9.9 An intermediate table links users and items.

We added a join table earlier in this chapter for a one-to-one association. To guarantee the multiplicity of one-to-one, we applied a unique constraint on a foreign key column of the join table. In the current case, we have a one-to-many multiplicity, so only the `ITEM_ID` primary key column has to be unique: only one `User` can buy any given `Item`s. The `BUYER_ID` column isn't unique because a `User` can buy many `Item`s. (The source code that follows can be found in the `onetomany-jointable` folder.)

The mapping of the `User#boughtItems` collection is simple:

```
Path: onetomany-jointable/src/main/java/com/manning/javapersistence/cl
➡ onetomany/jointable/User.java
```

```
@Entity
@Table(name = "USERS")
public class User {
    // . . .
    @OneToMany(mappedBy = "buyer")
    private Set<Item> boughtItems = new HashSet<>();
    // . . .
}
```

This is the usual read-only side of a bidirectional association, with the actual mapping to the schema on the “mapped by” side, the `Item#buyer`. It will be a clean, optional one-to-many/many-to-one relationship.

```
Path: onetomany-jointable/src/main/java/com/manning/javapersistence/cl
➡ onetomany/jointable/Item.java
```

```
@Entity
public class Item {
    // . . .
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinTable(
        name = "ITEM_BUYER",
        joinColumns =
            @JoinColumn(name = "ITEM_ID"),           Ⓐ
        inverseJoinColumns =
            @JoinColumn(nullable = false)           Ⓑ
    )
    private User buyer;
```

```
// . . .
}
```

Ⓐ If an `Item` hasn't been bought, there is no corresponding row in the `ITEM_BUYER` join table. The relationship will thus be optional. The join column is named `ITEM_ID` (it would default to `ID`).

Ⓑ The inverse join column will default to `BUYER_ID`, and it is not nullable.

We don't have any problematic nullable columns in our schema. Still, we should write a procedural constraint and a trigger that runs on `INSERT` for the `ITEM_BUYER` table: "Only allow insertion of a buyer if the auction end time for the given item has been reached and the user made the winning bid."

The next example is our last with one-to-many associations. So far, you've seen one-to-many associations from an entity to another entity. An embeddable component class may also have a one-to-many association to an entity, and this is what we'll deal with now.

9.2.4 One-to-many association in an embeddable class

Consider again the embeddable component mapping we've been repeating for a few chapters: the `Address` of a `User`. We'll now extend this example by adding a one-to-many association from `Address` to `Shipment`: a collection called `deliveries`. Figure 9.10 shows the UML class diagram for this model.

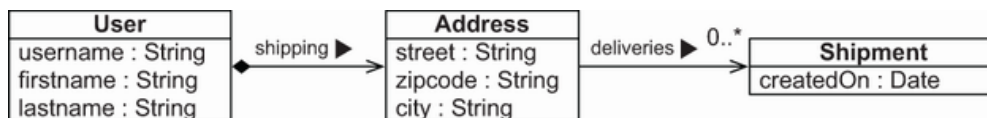


Figure 9.10 The one-to-many relationship from `Address` to `Shipment`

The `Address` is an `@Embeddable` class, not an entity. It can own a unidirectional association to an entity; here, it's a one-to-many multiplicity to `Shipment`. We'll look at an embeddable class having a many-to-one association with an entity in the next section. (The source code that follows can be found in the `onetomany-embeddable` folder.)

The `Address` class has a `Set<Shipment>` representing this association:

```
Path: onetomany-embeddable/src/main/java/com/manning/javapersistence/c
➡ onetomany/embeddable/Address.java
```

```

@Embeddable
public class Address {
    @NotNull
    @Column(nullable = false)
    private String street;
    @NotNull
    @Column(nullable = false, length = 5)
    private String zipcode;
    @NotNull
    @Column(nullable = false)
    private String city;
    @OneToMany
    @JoinColumn(
        name = "DELIVERY_ADDRESS_USER_ID",    ①
        nullable = false
    )
    private Set<Shipment> deliveries = new HashSet<>();
    // . . .
}

```

① The first mapping strategy for this association is with a `@JoinColumn` named `DELIVERY_ADDRESS_USER_ID` (it would default to `DELIVERIES_ID`).

This foreign key-constrained column is in the `SHIPMENT` table, as you can see in figure 9.11.

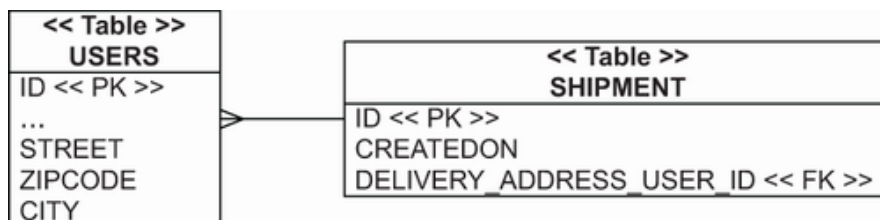


Figure 9.11 A primary key in the `USERS` table links the `USERS` and `SHIPMENT` tables.

Embeddable components don't have their own identifier, so the value in the foreign key column is the value of a `User`'s identifier, which embeds the `Address`. Here we also declare the join column `nullable = false`, so a `Shipment` must have an associated delivery address. Of course, bidirectional navigation isn't possible: the `Shipment` can't have a reference to the `Address` because embedded components can't have shared references.

If the association is optional and we don't want a nullable column, we can map the association to an intermediate join/link table, as shown in figure 9.12.

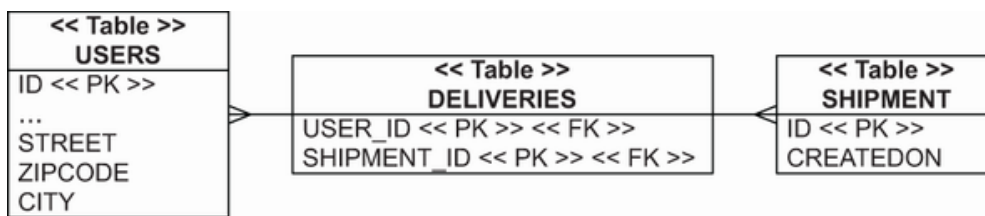


Figure 9.12 Using an intermediate table between `USERS` and `SHIPMENT` to represent an optional association

The mapping of the collection in `Address` now uses a `@JoinTable` instead of a `@JoinColumn`. (The source code that follows can be found in the `onetomany-embeddable-jointable` folder.)

Path: `onetomany-embeddable-jointable/src/main/java/com/manning`
 ➔ `/javapersistence/ch09/onetomany/embeddablejointable/Address.java`

```

@Embeddable
public class Address {
    @NotNull
    @Column(nullable = false)
    private String street;
    @NotNull
    @Column(nullable = false, length = 5)
    private String zipcode;
    @NotNull
    @Column(nullable = false)
    private String city;

    @OneToMany
    @JoinTable(
        name = "DELIVERIES",
        joinColumns =
            @JoinColumn(name = "USER_ID"),
        inverseJoinColumns =
            @JoinColumn(name = "SHIPMENT_ID")
    )
    private Set<Shipment> deliveries = new HashSet<>();
    // . . .
}
  
```

Ⓐ The name of the join table will be `DELIVERIES` (it would otherwise default to `USERS_SHIPMENT`).

Ⓑ The name of the join column will be `USER_ID` (it would otherwise default to `USERS_ID`).

Ⓒ The name of the inverse join column will be `SHIPMENT_ID` (it would otherwise default to `SHIPMENTS_ID`).

Note that if we declare neither `@JoinTable` nor `@JoinColumn`, the `@OneToMany` in an embeddable class defaults to a join table strategy.

From within the owning entity class, we can override property mappings of an embedded class with `@AttributeOverride`, as demonstrated in section 6.2.3. If we want to override the join table or column mapping of an entity association in an embeddable class, we can use `@AssociationOverride` in the owning entity class instead. We can't, however, switch the mapping strategy; the mapping in the embeddable component class decides whether a join table or join column is used.

A join table mapping is, of course, also applicable in true *many-to-many* mappings.

9.3 Many-to-many and ternary associations

The association between `Category` and `Item` is a *many-to-many* association, as you can see in figure 9.13. In a real system, we may not have a many-to-many association—our experience is that there is almost always other information that must be attached to each link between associated instances. Some examples are the timestamp when an `Item` was added to a `Category`, and the `User` responsible for creating the link. We'll expand the example later in this section to cover such cases, but we'll start with a regular and simpler many-to-many association.



Figure 9.13 A many-to-many association between `Category` and `Item`

9.3.1 Unidirectional and bidirectional many-to-many associations

A join table in the database represents a regular many-to-many association, which some developers also call a *link table* or *association table*.

Figure 9.14 shows a many-to-many relationship with a link table.

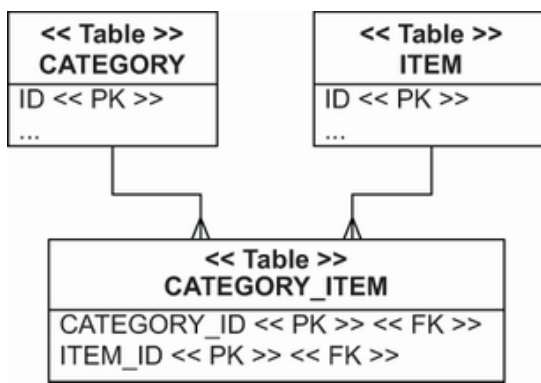


Figure 9.14 CategorizedItem is the link between Category and Item.

The `CATEGORY_ITEM` link table has two columns, both with foreign key constraints referencing the `CATEGORY` and `ITEM` tables, respectively. Its primary key is a composite key of both columns. We can only link a particular `Category` and `Item` once, but we can link the same item to several categories. The source code that follows can be found in the `many-to-many-bidirectional` folder.

In JPA, we map many-to-many associations with `@ManyToMany` on a collection:

Path: `manytomany-bidirectional/src/main/java/com/manning`
 ➔ `/javapersistence/ch09/manytomany/bidirectional/Category.java`

```

@Entity
public class Category {
    // . . .
    @ManyToMany(cascade = CascadeType.PERSIST)
    @JoinTable(
        name = "CATEGORY_ITEM",
        joinColumns = @JoinColumn(name = "CATEGORY_ID"),
        inverseJoinColumns = @JoinColumn(name = "ITEM_ID")
    )
    private Set<Item> items = new HashSet<>();
    // . . .
}
  
```

As usual, we can enable `CascadeType.PERSIST` to make it easier to save data. When we reference a new `Item` from the collection, Hibernate or Spring Data JPA using Hibernate makes it persistent. Let's make this association bidirectional (we don't have to if we don't need to):

Path: `manytomany-bidirectional/src/main/java/com/manning`
 ➔ `/javapersistence/ch09/manytomany/bidirectional/Item.java`

```

@Entity
public class Item {
  
```

```

        // . . .
        @ManyToMany(mappedBy = "items")
        private Set<Category> categories = new HashSet<>();
        // . . .
    }

```

As in any bidirectional mapping, one side is “mapped by” the other side. The `Item#categories` collection is effectively read-only; Hibernate will analyze the content of the `Category#items` side when storing data.

Next we’ll create two categories and two items and link them with many-to-many multiplicity:

```

Path: manytomany-bidirectional/src/test/java/com/manning
➡ /javapersistence/ch09/manytomany/bidirectional/TestService.java

```

```

Category someCategory = new Category("Some Category");
Category otherCategory = new Category("Other Category");
Item someItem = new Item("Some Item");
Item otherItem = new Item("Other Item");
someCategory.addItem(someItem);
someItem.addCategory(someCategory);
someCategory.addItem(otherItem);
otherItem.addCategory(someCategory);
otherCategory.addItem(someItem);
someItem.addCategory(otherCategory);
categoryRepository.save(someCategory);
categoryRepository.save(otherCategory);

```

Because we enabled transitive persistence, saving the categories makes the entire network of instances persistent. On the other hand, the cascading options `ALL`, `REMOVE`, and orphan deletion (discussed in section 8.3.3) aren’t meaningful for many-to-many associations. This is a good point for testing whether we understand entities and value types. Try to come up with reasonable answers as to why these cascading types don’t make sense for a many-to-many association. Hint: Think about what may happen if deleting a record will automatically delete a related record.

Could we use a `List` instead of a `Set`, or even a bag? The `Set` matches the database schema perfectly because there can be no duplicate links between `Category` and `Item`. A bag implies duplicate elements, so we would need a different primary key for the join table. Hibernate’s proprietary `@CollectionId` annotation can provide this, as demonstrated in section 8.1.5. However, one of the alternative many-to-many strategies we’ll discuss in a moment is a better choice if we need to support duplicate links.

We can map indexed collections such as a `List` with the regular `@ManyToMany`, but only on one side. Remember that in a bidirectional relationship, one side has to be “mapped by” the other side, meaning its value is ignored when Hibernate synchronizes with the database. If both sides are lists, we can only make the index of one side persistent.

A regular `@ManyToMany` mapping hides the link table; there is no corresponding Java class, only some collection properties. So whenever someone says, “My link table has more columns with information about the link” (and, in our experience, someone always says this sooner rather than later), we need to map this information to a Java class.

9.3.2 Many-to-many with an intermediate entity

We can always represent a many-to-many association as two many-to-one associations to an intervening class, and this is what we’ll do next. We won’t hide the link table; we’ll represent it with a Java class. This model is usually more easily extensible, so we tend not to use regular many-to-many associations in applications. It’s a lot of work to change the code later when more columns are inevitably added to a link table, so before mapping a `@ManyToMany` as shown in the previous section, consider the alternative shown in figure 9.15.



Figure 9.15 `CategorizedItem` will be the link between `Category` and `Item`.

Imagine that we need to record some information each time we add an `Item` to a `Category`. The `CategorizedItem` entity captures the timestamp and the user who created the link. This domain model requires additional columns on the join table, as you can see figure 9.16

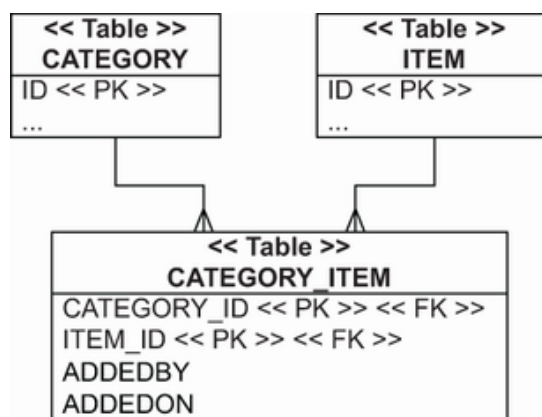


Figure 9.16 Additional columns on the join table in a many-to-many relationship

The `CategorizedItem` entity maps to the link table, as you'll see in listing 9.4. (The source code that follows can be found in the `manytomany-linkentity` folder.) This will involve a large chunk of code with some new annotations. First, it will be an immutable entity class (annotated with `@org.hibernate.annotations.Immutable`), so we'll never update properties after creation. Hibernate can make some optimizations, such as avoiding dirty checking during flushing of the persistence context, if we declare the class immutable.

The entity class will have a composite key, which we'll encapsulate in a static nested embeddable component class for convenience. The identifier property and its composite key columns will be mapped to the entity's table through the `@EmbeddedId` annotation.

Listing 9.4 Mapping a many-to-many relationship with `CategorizedItem`

```
Path: manytomany-linkentity/src/main/java/com/manning/javapersistence
➡ /ch09/manytomany/linkentity/CategorizedItem.java
```

```
@Entity
@Table(name = "CATEGORY_ITEM")
@org.hibernate.annotations.Immutable ①
public class CategorizedItem {
    @Embeddable
    public static class Id implements Serializable { ②
        @Column(name = "CATEGORY_ID")
        private Long categoryId;
        @Column(name = "ITEM_ID")
        private Long itemId;

        public Id() {
        }

        public Id(Long categoryId, Long itemId) {
            this.categoryId = categoryId;
            this.itemId = itemId;
        }
        //implementing equals and hashCode
    }
    @EmbeddedId ③
    private Id id = new Id();
    @Column(updatable = false)
    @NotNull
    private String addedBy; ④
    @Column(updatable = false)
    @NotNull
    @CreationTimestamp
    private LocalDateTime addedOn; ⑤
}
```

```

    @ManyToOne
    @JoinColumn(
        name = "CATEGORY_ID",
        insertable = false, updatable = false)
    private Category category;
    @ManyToOne
    @JoinColumn(
        name = "ITEM_ID",
        insertable = false, updatable = false)
    private Item item;
    public CategorizedItem(
        String addedByUsername,
        Category category,
        Item item) {
        this.addedBy = addedByUsername;
        this.category = category;
        this.item = item;
        this.id.categoryId = category.getId();
        this.id.itemId = item.getId();
        category.addCategorizedItem(this);
        item.addCategorizedItem(this);
    }
    // . . .
}

```

Ⓐ The class is immutable, as annotated with `@org.hibernate.annotations.Immutable`.

Ⓑ An entity class needs an identifier property. The primary key of the link table is the composite of `CATEGORY_ID` and `ITEM_ID`. We can externalize this `Id` class into its own file, of course.

Ⓒ The new `@EmbeddedId` annotation maps the identifier property and its composite key columns to the entity's table.

Ⓓ The basic property mapping the `addedBy` username to a column of the join table.

Ⓔ The basic property mapping the `addedOn` timestamp to a column of the join table. This is the “additional information about the link” that interests us.

Ⓕ The `@ManyToOne` property `category` is already mapped in the identifier.

Ⓖ The `@ManyToOne` property `item` is already mapped in the identifier. The trick here is to make them read-only, with the `updatable=false`, `insertable=false` settings. This means Hibernate or Spring Data JPA

using Hibernate writes the values of these columns by taking the identifier value of `CategorizedItem`. At the same time, we can read and browse the associated instances through `categorizedItem.getItem()` and `getCategory()`. (If we map the same column twice without making one mapping read-only, Hibernate or Spring Data JPA using Hibernate will complain on startup about a duplicate column mapping.)

Ⓜ We can also see that constructing a `CategorizedItem` involves setting the values of the identifier. The application always assigns composite key values; Hibernate doesn't generate them.

Ⓜ The constructor sets the `addedBy` field value and guarantees referential integrity by managing collections on both sides of the association.

Ⓜ The constructor sets the `categoryId` field value. We'll map these collections next to enable bidirectional navigation. This is a unidirectional mapping and enough to support the many-to-many relationship between `Category` and `Item`. To create a link, we instantiate and persist a `CategorizedItem`. If we want to break a link, we remove the `CategorizedItem`. The constructor of `CategorizedItem` requires that we provide already persistent `Category` and `Item` instances.

If bidirectional navigation is required, we can map a `@OneToMany` collection in `Category` and/or `Item`. Here it is in `Category`:

```
Path: manytomany-linkentity/src/main/java/com/manning/javapersistence
➡ /ch09/manytomany/linkentity/Category.java
```

```
@Entity
public class Category {
    // . . .
    @OneToMany(mappedBy = "category")
    private Set<CategorizedItem> categorizedItems = new HashSet<>();
    // . . .
}
```

Here it is in `Item`:

```
Path: manytomany-linkentity/src/main/java/com/manning
➡ /javapersistence/ch09/manytomany/linkentity/Item.java
```

```
@Entity
public class Item {
    // . . .
    @OneToMany(mappedBy = "item")
    private Set<CategorizedItem> categorizedItems = new HashSet<>();
}
```

```

    // . . .
}

```

Both sides are “mapped by” the annotations in `CategorizedItem`, so Hibernate already knows what to do when we iterate through the collection returned by either `getCategorizedItems()` method.

This is how we create and store links:

Path: `manytomany-linkentity/src/test/java/com/manning/javapersistence`
 ➡ `/ch09/manytomany/linkentity/TestService.java`

```

Category someCategory = new Category("Some Category");
Category otherCategory = new Category("Other Category");
categoryRepository.save(someCategory);
categoryRepository.save(otherCategory);
Item someItem = new Item("Some Item");
Item otherItem = new Item("Other Item");
itemRepository.save(someItem);
itemRepository.save(otherItem);
CategorizedItem linkOne = new CategorizedItem(
    "John Smith", someCategory, someItem
);
CategorizedItem linkTwo = new CategorizedItem(
    "John Smith", someCategory, otherItem
);
CategorizedItem linkThree = new CategorizedItem(
    "John Smith", otherCategory, someItem
);
categorizedItemRepository.save(linkOne);
categorizedItemRepository.save(linkTwo);
categorizedItemRepository.save(linkThree);

```

The primary advantage of this strategy is the possibility of bidirectional navigation: we can get all items in a category by calling `someCategory.getCategorizedItems()`, and we can also navigate from the opposite direction with `someItem.getCategorizedItems()`. A disadvantage is the more complex code needed to manage the `CategorizedItem` entity instances to create and remove links, which we have to save and delete independently. We also need some infrastructure in the `CategorizedItem` class, such as the composite identifier. One small improvement would be to enable `CascadeType.PERSIST` on some of the associations, reducing the number of calls to `save()`.

In this example, we stored the user who created the link between `Category` and `Item` as a simple name string. If the join table instead had a foreign key column called `USER_ID`, we’d have a ternary relation-

ship. The `CategorizedItem` would have a `@ManyToOne` for `Category`, `Item`, and `User`.

In the following section, we'll demonstrate another many-to-many strategy. To make it a bit more interesting, we'll make it a ternary association.

9.3.3 Ternary associations with components

In the previous section, we represented a many-to-many relationship with an entity class mapped to a link table. A potentially simpler alternative is mapping to an embeddable component class. The source code that follows can be found in the `manytomany-ternary` folder.

```
Path: manytomany-ternary/src/main/java/com/manning/javapersistence
➡ /ch09/manytomany/ternary/CategorizedItem.java
```

```
@Embeddable
public class CategorizedItem {
    @ManyToOne
    @JoinColumn(
        name = "ITEM_ID",
        nullable = false, updatable = false
    )
    private Item item;
    @ManyToOne
    @JoinColumn(
        name = "USER_ID",
        updatable = false
    )
    @NotNull Ⓐ
    private User addedBy;
    @Column(updatable = false)
    @NotNull Ⓐ
    private LocalDateTime addedOn = LocalDateTime.now();
    public CategorizedItem() {
    }
    public CategorizedItem(User addedBy,
                           Item item) {
        this.addedBy = addedBy;
        this.item = item;
    }
    // . . .
}
```

Ⓐ The `@NotNull` annotations do not generate an SQL constraint, so the annotated fields will not be part of the primary key.

The new mappings here are `@ManyToOne` associations in an `@Embeddable` and the additional foreign key join column `USER_ID`, making this a

ternary relationship. Look at the database schema in figure 9.17.

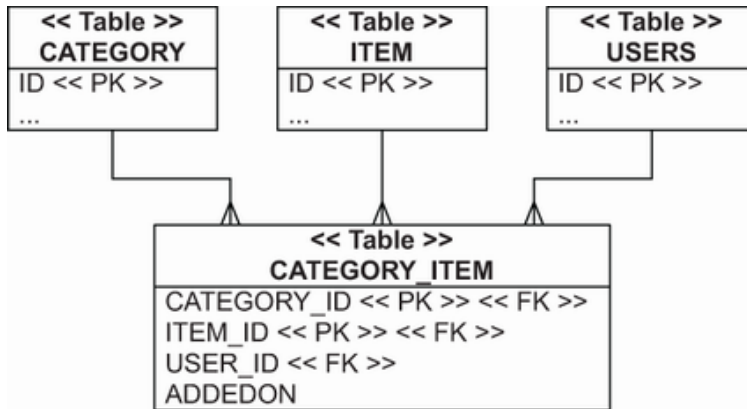


Figure 9.17 A link table with three foreign key columns

The owner of the embeddable component collection is the `Category` entity:

```
Path: manytomany-ternary/src/main/java/com/manning/javapersistence
➡ /ch09/manytomany/ternary/Category.java
```

```
@Entity
public class Category {
    // . . .
    @ElementCollection
    @CollectionTable(
        name = "CATEGORY_ITEM",
        joinColumns = @JoinColumn(name = "CATEGORY_ID")
    )
    private Set<CategorizedItem> categorizedItems = new HashSet<>();
    // . . .
}
```

Unfortunately, this mapping isn't perfect: when we map an `@ElementCollection` of embeddable type, all properties of the target type that are `nullable=false` become part of the (composite) primary key. We want all columns in `CATEGORY_ITEM` to be `NOT NULL`. Only the `CATEGORY_ID` and `ITEM_ID` columns should be part of the primary key, though. The trick is to use the Bean Validation `@NotNull` annotation on properties that shouldn't be part of the primary key. In that case (because it's an embeddable class), Hibernate ignores the Bean Validation annotation for primary key realization and SQL schema generation. The downside is that the generated schema won't have the appropriate `NOT NULL` constraints on the `USER_ID` and `ADDEDON` columns, which we should fix manually.

The advantage of this strategy is the implicit lifecycle of the link components. To create an association between a `Category` and an `Item`, add a

new `CategorizedItem` instance to the collection. To break the link, remove the element from the collection. No extra cascading settings are required, and the Java code is simplified (albeit spread over more lines):

```
Path: manytomany-ternary/src/test/java/com/manning/javapersistence
➡ /ch09/manytomany/ternary/TestService.java
```

```
Category someCategory = new Category("Some Category");
Category otherCategory = new Category("Other Category");
categoryRepository.save(someCategory);
categoryRepository.save(otherCategory);
Item someItem = new Item("Some Item");
Item otherItem = new Item("Other Item");
itemRepository.save(someItem);
itemRepository.save(otherItem);
User someUser = new User("John Smith");
userRepository.save(someUser);
CategorizedItem linkOne = new CategorizedItem(
    someUser, someItem
);
someCategory.addCategorizedItem(linkOne);
CategorizedItem linkTwo = new CategorizedItem(
    someUser, otherItem
);
someCategory.addCategorizedItem(linkTwo);
CategorizedItem linkThree = new CategorizedItem(
    someUser, someItem
);
otherCategory.addCategorizedItem(linkThree);
```

There is no way to enable bidirectional navigation: an embeddable component, such as `CategorizedItem`, by definition can't have shared references. We can't navigate from `Item` to `CategorizedItem`, and there is no mapping of this link in `Item`. Instead, we can write a query to retrieve the categories, given an `Item`:

```
Path: manytomany-ternary/src/test/java/com/manning/javapersistence
➡ /ch09/manytomany/ternary/TestService.java
```

```
List<Category> categoriesOfItem =
    categoryRepository.findCategoryWithCategorizedItems(item1);
assertEquals(2, categoriesOfItem.size());
```

The `findCategoryWithCategorizedItems` method is annotated with the `@Query` annotation:

```
Path: manytomany-ternary/src/main/java/com/manning/javapersistence
➡ /ch09/repositories/manytomany/ternary/CategoryRepository.java
```

```
@Query("select c from Category c join c.categorizedItems ci where
        ci.item = :itemParameter")
List<Category> findCategoryWithCategorizedItems(
    @Param("itemParameter") Item itemParameter);
```

We've now completed our first ternary association mapping. In the previous chapters, we saw ORM examples with maps; the keys and values of those maps were always of basic or embeddable type. In the following section, we'll use more complex key/value pair types and their mappings.

9.4 Entity associations with maps

Map keys and values can be references to other entities, providing another strategy for mapping many-to-many and ternary relationships. First, let's assume that only the value of each map entry is a reference to another entity.

9.4.1 One-to-many with a property key

If the value of each map entry is a reference to another entity, we have a one-to-many entity relationship. The key of the map is of a basic type, such as a `Long` value. (The source code that follows can be found in the `maps-mapkey` folder.)

An example of this structure would be an `Item` entity with a map of `Bid` instances, where each map entry is a pair consisting of `Bid` identifier and reference to a `Bid` instance. When we iterate through `someItem.getBids()`, we iterate through map entries that look like `(1, <reference to Bid with PK 1>)`, `(2, <reference to Bid with PK 2>)`, and so on:

```
Path: maps-mapkey/src/test/java/com/manning/javapersistence
➡ /ch09/maps/mapkey/TestService.java
```

```
Item item = itemRepository.findById(someItem.getId()).get();
assertEquals(2, item.getBids().size());
for (Map.Entry<Long, Bid> entry : item.getBids().entrySet()) {
    assertEquals(entry.getKey(), entry.getValue().getId());
}
```

The underlying tables for this mapping are nothing special; we have `ITEM` and `BID` tables, with an `ITEM_ID` foreign key column in the `BID` table. This is the same schema as demonstrated in figure 8.14 for a one-to-

many/many-to-one mapping with a regular collection instead of a `Map`. Our motivation here is a slightly different representation of the data in the application.

In the `Item` class, we'll include a `Map` property named `bids`:

```
Path: maps-mapkey/src/main/java/com/manning/javapersistence
➡ /ch09/maps/mapkey/Item.java
```

```
@Entity
public class Item {
    // . . .
    @MapKey(name = "id")
    @OneToMany(mappedBy = "item")
    private Map<Long, Bid> bids = new HashMap<>();
    // . . .
}
```

New here is the `@MapKey` annotation. It maps a property of the target entity: in this case, the `Bid` entity, as the key of the map. The default if we omit the `name` attribute is the identifier property of the target entity, so the `name` option here is redundant. Because the keys of a map form a set, we should expect values to be unique for a particular map. This is the case for `Bid` primary keys but likely not for any other property of `Bid`. It's up to us to ensure that the selected property has unique values—Hibernate or Spring Data JPA using Hibernate won't check.

The primary and rare use case for this mapping technique is to iterate map entries with some property of the entry entity value as the entry key, maybe because it's convenient for how we'd like to render the data. A more common situation is a map in the middle of a ternary association.

9.4.2 Key/value ternary relationship

You may be a little bored by now with all the mapping experiments that we've executed, but we promise this is the last time we'll show another way of mapping the association between `Category` and `Item`.

Previously, in section 9.3.3, we used an embeddable `CategorizedItem` component to represent the link. Here we'll show a representation of the relationship with a `Map` instead of an additional Java class. The key of each map entry is an `Item`, and the related value is the `User` who added the `Item` to the `Category`, as shown in figure 9.18.

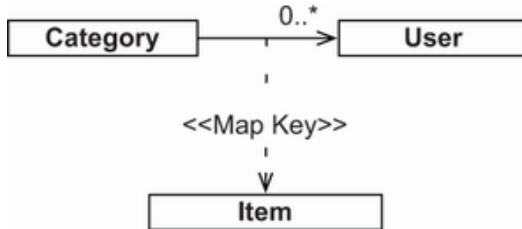


Figure 9.18 A Map with entity associations as key/value pairs

The link/join table in the schema, as you can see in figure 9.19, has three columns: `CATEGORY_ID`, `ITEM_ID`, and `USER_ID`. The Map is owned by the `Category` entity. The source code that follows can be found in the `maps-ternary` folder.

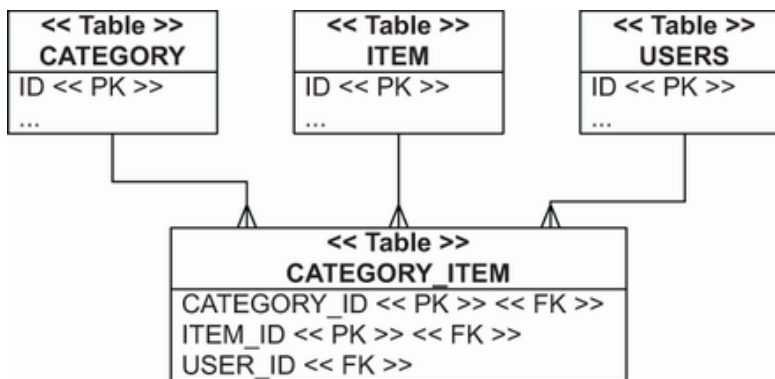


Figure 9.19 The link table represents the map key/value pairs.

The following code models the relationship between `Category` and `Item` using a map.

Path: `maps-ternary/src/main/java/com/manning/javapersistence`
 ➔ `/ch09/maps/ternary/Category.java`

```

@Entity
public class Category {
    // . . .
    @ManyToMany(cascade = CascadeType.PERSIST)
    @MapKeyJoinColumn(name = "ITEM_ID")
    @JoinTable(
        name = "CATEGORY_ITEM",
        joinColumns = @JoinColumn(name = "CATEGORY_ID"),
        inverseJoinColumns = @JoinColumn(name = "USER_ID")
    )
    private Map<Item, User> itemAddedBy = new HashMap<>();
    // . . .
}
  
```

Ⓐ The `@MapKeyJoinColumn` is optional; Hibernate or Spring Data JPA using Hibernate will default to the column name `ITEMADDED_BY_KEY` for

the join/foreign key column referencing the `ITEM` table.

To create a link between all three entities, all instances must already be in persistent state and then put into the map:

```
Path: maps-ternary/src/test/java/com/manning/javapersistence
➡ /ch09/maps/ternary/TestService.java
```

```
someCategory.putItemAddedBy(someItem, someUser);
someCategory.putItemAddedBy(otherItem, someUser);
otherCategory.putItemAddedBy(someItem, someUser);
```

To remove the link, remove the entry from the map. This manages a complex relationship, hiding a database link table with three columns. But remember that, in practice, link tables often grow additional columns, and changing all the Java application code later is expensive if you depend on a `Map` API. Earlier, we had an `ADDEDON` column with a timestamp when the link was created, but we had to drop it for this mapping.

Summary

- Complex entity associations can be mapped using one-to-one associations, one-to-many associations, many-to-many associations, ternary associations, and entity associations with maps.
- You can create one-to-one associations by sharing a primary key, using a foreign primary key generator, using a foreign key join column, or using a join table.
- You can create one-to-many associations by considering one-to-many bags, using unidirectional and bidirectional list mappings, applying optional one-to-many with a join table, or creating one-to-many associations in an embeddable class.
- You can create unidirectional and bidirectional many-to-many associations and many-to-many associations with an intermediate entity.
- You can build ternary associations with components and entity associations with maps.
- You can often best represent many-to-many entity associations as two many-to-one associations from an intermediate entity class or with a collection of components.
- Before attempting a complex collection mapping, always make sure you actually need a collection. Ask yourself whether you frequently iterate through its elements.
- The Java structures used in this chapter may sometimes make data access easier, but typically they complicate data storage, updates, and deletion.

